

Effect of Input Feeding in Attention-based Sequence-to-Sequence Translation

Jeonghoon Park (20201118)
UNIST
hoonably@unist.ac.kr

To analyze the effect of input feeding, I compared two global attention models trained for 3 epochs under the same setting. The best checkpoint was selected based on validation loss, and the learning rate was set to $1e-3$.

1 Experimental Results

Attention	Input Feeding	Best Validation Loss	BLEU
Global	No	1.8376	0.1806
Global	Yes	1.7863	0.2015

Table 1: Comparison of global attention models with and without input feeding.

Qualitative Example

Table 2 shows one validation example comparing the generated Korean outputs.

Without Input Feeding	With Input Feeding	Answer
5059 알루미늄 합금은 주로 마그네슘과 함께 건조된 알루미늄-매그네슘 합금입니다. 열처리에 의해 강화되지 않아 재료의 강화 또는 냉기계적 작동으로 인해 더욱 강해졌습니다. 열처리가 호전되면 5059년 용기에 용기된 것으로 간주되지는 못이 가능하며, 대부분의 기계적 강도는 기계적 강도를 초래할 수 있습니다. 5059개의 합금은 1999년 코러스 알루미늄에서 연구자들이 호스루늄의 연구자들에 의해 느슨하게 관련되어 있습니다.	5059 알루미늄 합금은 주로 마그네슘으로 간주됩니다. 열처리로 인해 강화되지 않고, 대신 빗물의 냉소 정비 또는 냉소한 기계적 작동으로 인해 강수량이 높아지는 경우가 많습니다. 열처리가 열성 대우가 발생하면 5059년은 기계적 강도 대부분을 쉽게 감지하고 유지하기 위해 나타났습니다. 5059 합금은 1999년 코러스 알루미늄에서 연구원들에 의해 균적으로 관련되어 있습니다.	5059 알루미늄 합금은 주로 마그네슘으로 합금된 알루미늄-마그네슘 합금입니다. 열처리로 강화되지 않고, 대신 재료의 변형 경화 또는 냉간 기계 가공으로 강해집니다. 열처리가 강도에 큰 영향을 미치지 않기 때문에 5059는 용접이 용이하고 기계적 강도를 대부분 유지할 수 있습니다. 5059 합금은 1999년 코러스 알루미늄의 연구원들에 의해 밀접하게 관련된 5083 알루미늄 합금에서 유래했습니다.

Table 2: Qualitative comparison of generated Korean translations with and without input feeding.

The model with input feeding achieved a lower validation loss and a higher BLEU score. Compared to the model without input feeding, the validation loss decreased from **1.8376** to **1.7863**, and the BLEU score increased from **0.1806** to **0.2015**. This corresponds to an absolute BLEU improvement of **0.0209**.

The qualitative example shows a similar trend. The model without input feeding contains repeated and unnatural phrases such as “5059년 용기에 용기된” and “기계적 강도는 기계적 강도를”. The input feeding model also contains mistranslations, but it better preserves some key source-side concepts such as magnesium, heat treatment, and mechanical strength. Although both outputs are imperfect, the input feeding output is slightly more coherent and closer to the reference translation.

2 Discussion

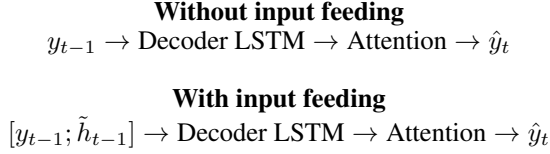


Figure 1: Conceptual comparison of decoding with and without input feeding. Here, $\tilde{h}_{t-1}$ denotes the attentional output from the previous decoding step.

The results show that input feeding improves the translation quality in this setting. In the model without input feeding, the decoder predicts each target token using the current target-side input embedding and its recurrent hidden state. Although the hidden state can carry information from previous decoding steps, the previous attention decision is only preserved implicitly through the recurrent state.

In contrast, input feeding explicitly passes the attentional output from the previous decoding step to the next decoding step. Therefore, the decoder input contains not only the current target token embedding, but also a summary of the recent source-side context selected by attention. This gives the decoder more direct access to its previous alignment behavior and helps maintain continuity across decoding steps.

This mechanism is useful for translation because the decoder needs to decide both what target word to generate and which source-side information to focus on. Without input feeding, the attention module may make less consistent alignment decisions across time steps, which can lead to repeated phrases, missing content, or less coherent translations. With input feeding, the previous attentional output acts as an additional short-term memory of the translation context, helping the decoder produce more stable next-token predictions.

The quantitative results are consistent with this interpretation. The model with input feeding achieved a lower validation loss and a higher BLEU score than the model without input feeding. The validation loss decreased from **1.8376** to **1.7863**, while BLEU increased from **0.1806** to **0.2015**. This corresponds to an absolute BLEU improvement of **0.0209**, or approximately an **11.6% relative improvement** over the model without input feeding.

The qualitative example also supports this trend, although the improvement is not perfect. The model without input feeding produces repeated and unnatural phrases, while the input feeding model preserves some important source-side concepts more clearly, such as magnesium, heat treatment, and mechanical strength. However, the input feeding output still contains mistranslations and misses some reference details. Overall, it is slightly more coherent and semantically closer to the reference translation.

Overall, the improvement is moderate but consistent across validation loss, BLEU, and the qualitative comparison. This suggests that input feeding provides a meaningful benefit to the LSTM-based sequence-to-sequence translation model with global attention by helping the decoder maintain better continuity of attentional context during generation.

Appendix: Additional Implementation Notes

Besides the required implementation parts, I made two additional implementation-related changes outside the specified TODO blocks and adjusted several experimental settings. These changes were not intended to artificially improve the model performance. Instead, they were necessary to make the training and evaluation procedures consistent with the autoregressive sequence-to-sequence formulation and to obtain meaningful experimental comparisons.

A Decoder Input Construction in `collate_fn`

The original `collate_fn` directly used the tokenized Korean target sentence as the decoder input. However, the decoder in a sequence-to-sequence model should start generation from a beginning-of-sentence token. During inference, the `generate` function starts decoding from the BOS token, so the training procedure should follow the same input format.

To fix this mismatch, I changed the target-side preprocessing so that the decoder input becomes:

[BOS, y_1 , y_2 , ..., y_n]

The labels are constructed from the same sequence, and padded positions are replaced with -100 so that they are ignored by the loss function. This allows the model to learn the correct teacher-forcing objective:

BOS → y_1
 y_1 → y_2
 y_2 → y_3
...

Without this change, the model would not be explicitly trained to predict the first target token from the BOS token, even though generation starts from BOS during evaluation. Therefore, this modification aligns the training and inference procedures.

```
def collate_fn(batch):
    english_corpus = [item["english"] for item in batch]
    korean_corpus = [item["korean"] for item in batch]
    english_corpus = tokenizer(english_corpus, padding=True, truncation=True, return_tensors="pt", max_length=512, pad_to_multiple_of=8)
    korean_corpus = tokenizer(korean_corpus, padding=True, truncation=True, return_tensors="pt", max_length=512, pad_to_multiple_of=8)

    labels = korean_corpus["input_ids"].clone()
    labels[korean_corpus["attention_mask"].eq(0)] = -100

    return {
        "encoder_input_ids": english_corpus["input_ids"],
        "encoder_attention_mask": english_corpus["attention_mask"],
        "decoder_input_ids": korean_corpus["input_ids"],
        "labels": korean_corpus["input_ids"],
    }

def collate_fn(batch):
    english_corpus = [item["english"] for item in batch]
    korean_corpus = [item["korean"] for item in batch]
    english_corpus = tokenizer(english_corpus, padding=True, truncation=True, return_tensors="pt", max_length=512, pad_to_multiple_of=8)
    korean_corpus = tokenizer(korean_corpus, padding=True, truncation=True, return_tensors="pt", max_length=512, pad_to_multiple_of=8)

    # labels = korean_corpus["input_ids"].clone()
    # labels[korean_corpus["attention_mask"].eq(0)] = -100

    batch_size, target_len = korean_corpus["input_ids"].shape
    decoder_input_ids = torch.full(
        (batch_size, target_len + 1),
        tokenizer.pad_token_id,
        dtype=korean_corpus["input_ids"].dtype,
    )
    decoder_input_ids[:, 0] = tokenizer.bos_token_id
    decoder_input_ids[:, 1:] = korean_corpus["input_ids"]

    labels = decoder_input_ids.clone()
    labels[:, 0] = -100
    labels[:, 1:] = korean_corpus["input_ids"]
    labels[korean_corpus["attention_mask"].eq(0)] = -100

    return {
        "encoder_input_ids": english_corpus["input_ids"],
        "encoder_attention_mask": english_corpus["attention_mask"],
        "decoder_input_ids": decoder_input_ids,
        "labels": labels,
    }
```

Figure 2: Modified decoder input construction in `collate_fn`.

B Restoring Ignored Label Positions Before BLEU Evaluation

For training, padded label positions are set to -100 because PyTorch’s cross-entropy loss ignores this value. However, -100 is not a valid token id for the tokenizer. If labels containing -100 are directly passed to `tokenizer.batch_decode`, the reference sentences used for BLEU computation can be decoded incorrectly.

To prevent this problem, I replaced -100 values with the tokenizer’s padding token id before decoding the reference sentences:

```
answer_ids = batch["labels"].masked_fill(
    batch["labels"].eq(-100),
    tokenizer.pad_token_id
)
ans = tokenizer.batch_decode(answer_ids, skip_special_tokens=True)
```

Since `skip_special_tokens=True` is used, the padding tokens are removed from the decoded reference sentences. This ensures that BLEU is computed using valid target translations.

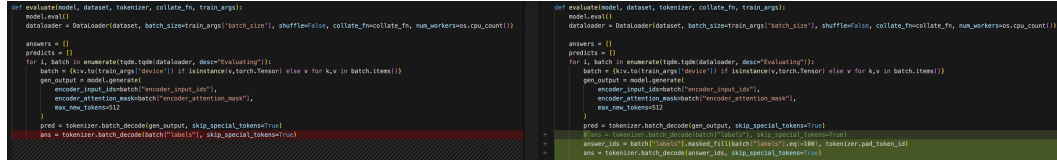


Figure 3: Restoring ignored label positions before BLEU evaluation.

C Local Attention Window Size

For the local attention experiment, I compared the default window size, `window_size=10`, with a larger window size, `window_size=64`. The result for `window_size=10` was obtained from a separate ablation run, while the result for `window_size=64` corresponds to the local attention model used in the main comparison. This comparison was included to explain why I used a larger local attention window in the main experiments.

Window Size	Best Validation Loss	BLEU
10	3.7329	0.0129
64	1.9850	0.1798

Table 3: Effect of local attention window size.

With the default window size of 10, the model produced highly repetitive and inaccurate translations. For example, the generated outputs repeatedly contained generic phrases such as repeated descriptions of fibers, remote areas, or locations, while failing to preserve the main content of the reference sentences. This behavior indicates that the attention window was too narrow for the model to access enough relevant source-side context during decoding.

After increasing the window size to 64, the validation loss decreased from **3.7329** to **1.9850**, and the BLEU score increased from **0.0129** to **0.1798**. The generated translations were still not perfect, but they became much more related to the reference sentences. For example, in the aluminum alloy example, the model correctly generated content related to the 5059 aluminum alloy, magnesium, heat treatment, and mechanical strength. In the animal sanctuary example, the model also recovered important entities and context such as Thornberry Animal Sanctuary, South Yorkshire, Rotherham, and Steve Bamford.

This suggests that the original local attention window was too small for this translation setting. A larger window allowed the decoder to attend to a wider source context, which reduced degenerate repetitive outputs and substantially improved both validation loss and BLEU. This change was not used to analyze the effect of input feeding, since the input feeding comparison in the main section was conducted only between the two global attention models.

D Precision and Compilation Settings

The original training code included the following optional lines:

```
model = model.to(torch.bfloat16)
model.compile()
```

I did not use these options in the final experiments. When these lines were enabled, PyTorch produced warnings related to the LSTM module weights not being stored as a single contiguous chunk of memory:

```
UserWarning: RNN module weights are not part of single contiguous chunk
of memory. This means they need to be compacted at every call, possibly
greatly increasing memory usage. To compact weights again call
flatten_parameters().
```

In practice, enabling these options made training slower in my environment and did not improve the model performance. Since the model could be trained without memory issues using the default precision, I kept the model in standard FP32 precision and did not apply `torch.compile()`.

The final model initialization was therefore:

```
model = Seq2Seq(config).to(training_config["device"])
# model = model.to(torch.bfloat16)
# model.compile()
```

This change was treated as an execution setting rather than a model architecture change. The same setting was used consistently for the compared runs.